Arrays

- An **array** is a collection of elements stored in **contiguous memory locations**.

Index

```
0     1     2     3     4
+-----+----+-----+-----+----+
| 12 | 35 | 17 | 99 | 28|
+-----+----+-----+-----+----+
```

Each element occupies the same amount of memory.

Arrays

Characteristics

- Fixed-size in most programming languages
- Fast indexing
- Stored in contiguous memory
- Efficient cache utilization

Accessing Elements

A = [12, 35, 17, 99, 28]

A[0] = 12

A[3] = 99

Access time is O(1)

Address = Base Address + (Index × Size of element)

Base Address = 1000

Integer size = 4 bytes

Find address of A[3]

1000 + (3 × 4)

= 1012

Arrays in Python

- Python does not have built-in fixed-size arrays like C.
- Instead, it provides
- list (dynamic array)
- array module
- NumPy arrays

```
from array import array

numbers = array('i', [10, 20, 30, 40])

print(numbers)
print(numbers[2])
```

'i' → integer

'f' → float

'd' → double

'b' → signed char

Insert

```
from array import array

a = array('i', [10,20,30])

a.append(40)

print(a)
```

Remove

```
from array import array

a = array('i', [10,20,30,40])

a.remove(20)

print(a)
```

Time Complexity

Operation	Complexity
Access	O(1)
Search	O(n)
Insert at end	O(1) (amortized)
Insert in middle	O(n)
Delete	O(n)

Python Lists

- • Dynamic arrays
- • Heterogeneous data
- • append(): amortized O(1)
- • Fast indexing

```
data = [10, "Python", 3.14, True]
```

Example

```
numbers = []

numbers.append(10)
numbers.append(20)
numbers.append(30)

print(numbers)
```

Internal Memory

Capacity = 4

```
+----+----+----+----+
|10 |20 |30 |    |
+----+----+----+----+
```

append(40)

```
+----+----+----+----+
|10 |20 |30 |40 |
+----+----+----+----+
```

Old

```
+----+----+----+----+
|10 |20 |30 |40 |
+----+----+----+----+
```

↓

New

```
+----+----+----+----+----+----+----+----+
|10 |20 |30 |40 |50 |    |    |    |
+----+----+----+----+----+----+----+----+
```

This resizing makes append() **amortized O(1)**.

List Operations

- append, insert, remove, pop, slicing, iteration

Create

numbers = [5,10,15]

append

numbers.append(20)

Insert

numbers.insert(1,50)

Remove

numbers.remove(10)

Delete

numbers.pop()

Removes last element.

Indexing

numbers[2]

Slicing

numbers[1:4]

Iteration

```
for item in numbers:
    print(item)
```

Membership

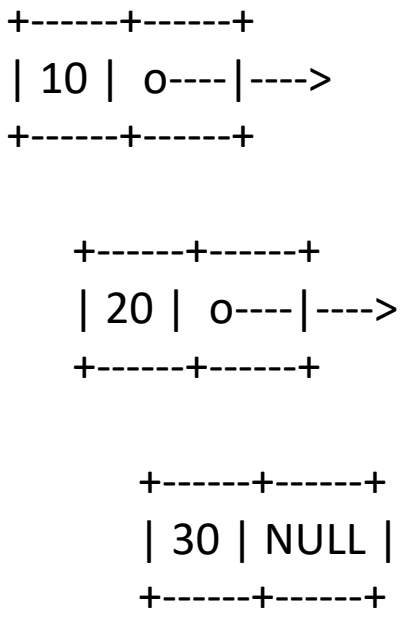
20 in numbers

Time Complexity

Operation	Complexity
Append	O(1) amortized
Access	O(1)
Search	O(n)
Delete	O(n)
Insert middle	O(n)

Linked Lists

- Node = data + next
- Not contiguous
- Sequential access



Unlike arrays,
linked lists do **not** store elements continuously.

Creating a Node

```
class Node:

    def __init__(self, data):

        self.data = data

        self.next = None
```

Creating Linked List

```
class Node:

    def __init__(self, data):

        self.data = data

        self.next = None

head = Node(10)

second = Node(20)

third = Node(30)

head.next = second

second.next = third
```

Traversing

```
current = head

while current:

    print(current.data)

    current = current.next
```

Inserting at Beginning

new = Node(5)

new.next = head

head = new

Insert at End

```
new = Node(40)

current = head

while current.next:

    current = current.next

current.next = new
```

Delete First Node

head = head.next

Delete Last Node

10 → 20 → 30 → 40

30

↓

30.next = None

Search

```
key = 20

current = head

while current:

    if current.data == key:

        print("Found")

        break

    current = current.next
```

Complete Linked List Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

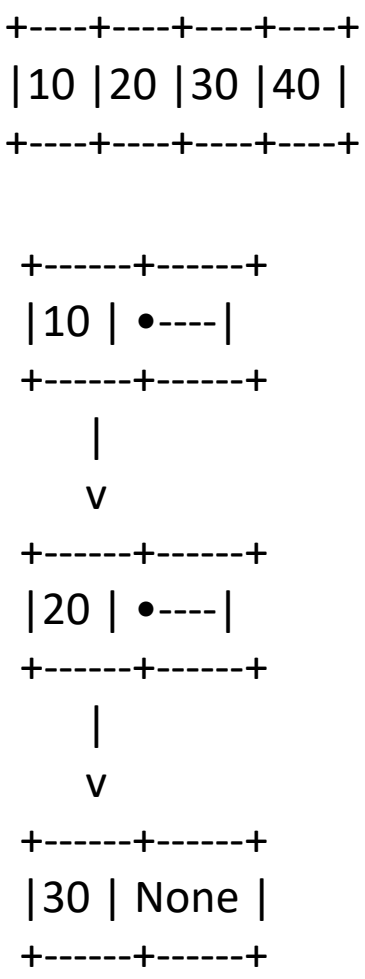
class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

ll = LinkedList()
ll.append(10)
ll.append(20)
ll.append(30)
ll.display()
```

Memory Comparison



Complexity Comparison

- Array/List: O(1) index
- Linked List: O(n) index
- Insert front: Linked List O(1)

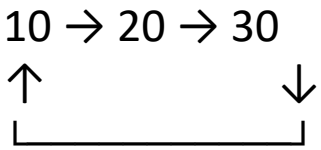
When to Use Which

- List: general purpose
- Array: typed numeric storage
- Linked List: frequent insert/delete

Doubly Linked List

Previous	NULL
Data	←10⇄20⇄30→
Next	NULL

Circular Linked List



Useful in

- CPU scheduling
- Round-robin algorithms
- Multiplayer games
- Music playlists

Complexity Comparison

Operation	Array	Python List	Linked List
Access by index	O(1)	O(1)	O(n)
Search	O(n)	O(n)	O(n)
Insert at beginning	O(n)	O(n)	O(1)
Insert at end	O(1)*	O(1) amortized	O(n)**
Delete at beginning	O(n)	O(n)	O(1)
Delete at end	O(1)*	O(1)	O(n)**
Extra memory	Low	Moderate	Higher (pointer per node)

Choosing the Right Data Structure

Use Case	Recommended Data Structure	Reason
Frequent random access by index	Array or Python List	Constant-time indexing
Frequent appending	Python List	Dynamic resizing with amortized O(1) append
Frequent insertion/deletion at the front	Linked List	No shifting of elements
Frequent insertion/deletion in the middle (with a reference to the node)	Linked List	Pointer updates are efficient
Numerical computing	Arrays (e.g., array module or NumPy arrays)	Compact storage and efficient computation
Mixed data types	Python List	Can store heterogeneous objects

Summary

- An **array** stores elements in **contiguous memory**, enabling very fast indexing but making insertions and deletions expensive due to element shifting.
- A **Python list** is implemented as a **dynamic array**. It provides automatic resizing, fast indexing, and efficient appends, making it the default choice for most Python programs.
- A **linked list** stores elements as **nodes connected by references (pointers)**. It excels at insertions and deletions once the position is known but has slower sequential access because elements are not stored contiguously.